p0019R7
Atomic Ref

Published Proposal, 6 May 2018

This version:
https://github.com/ORNL/cpp-proposals-pub/blob/master/P0019/P0019r7.html

Authors:
H. Carter Edwards
Hans Boehm
Olivier Giroux
Daniel Sunderland
Mark Hoemmen
David Hollman
Bryce Adelstein Lelbach
Jens Maurer

Audience:
SG1, LWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Extension to the atomic operations library to allow atomic operations to apply to non-atomic objects.

Table of Contents

§ 1. Revision History

[P0019r7]

- Update to reflect Jacksonville LWG review
- Update to reference resolution of padding bits from [P0528r2]
- Add a note clarifying that `atomic_ref` might not be lock free even if `atomic` is lock free
- Add wording for all member functions and specializations (in previous version only the constructor had wording)
- Added reference implementation
- targeted towards IS20
- Convert to bikeshed

[P0019r6]

- 2017-11-07 Albuquerque LEWG review
  - Settle on name `atomic_ref`
  - Split out atomic_ref into a separate paper, apply editorial changes accordingly
  - Restore copy constructor; not assignment operator
  - add **Throws: Nothing** to constructor but do not add noexcept
  - Remove *wrapping* terminology
  - Address problem of CAS on `atomic_ref<T>` where `T` is a struct containing padding bits
  - With these revisions move to LWG

[P0019r5]

- 2017-03-01 Kona LEWG review
  - Merge in P0440 Floating Point Atomic View because LEWG consensus to move P0020 Floating Point Atomic to C++20 IS
  - Rename from `atomic_view` and `atomic_array_view`; authors' selection `atomic_ref<T>` and `atomic_ref<T[]>`, other name suggested `atomic_wrapper`.
  - Remove `constexpr` qualification from default constructor because this qualification constrains implementations and does not add apparent value.
- Remove default constructor, copy constructor, and assignment operator for tighter alignment with `atomic<T>` and prevent empty references.
- Revise syntax to align with [P0558r1], Resolving atomic base class inconsistencies
- Recommend feature next macro

[P0019r4]

- wrapper constructor strengthen requires clause and omit throws clause
- Note types must be trivially copyable, as required for all atomics
- 2016-11-09 Issaquah SG1 decision: move to LEWG targeting Concurrency TS V2

[P0019r3]

- Align proposal with content of corresponding sections in N5131, 2016-07-15.
- Remove the *one root wrapping constructor* requirement from `atomic_array_view`.
- Other minor revisions responding to feedback from SG1 @ Oulu.

§ 2. Overview

This paper proposes an extension to the atomic operations library **[atomics]** to allow atomic operations to apply to non-atomic objects. As required by **[atomics.types.generic]** the value type **T** must be trivially copyable.

This paper includes *atomic floating point* capability defined in [P0020r5].

> Note: A reference implementation is available that works on compilers which support the GNU atomic builtin functions including recent versions of g++, icpc, and clang++. --end note

This paper is currently targeting C++20.

3. Motivation

3.1. Atomic Operations on a Single Non-atomic Object

An *atomic reference* is used to perform atomic operations on a referenced non-atomic object. The intent is for *atomic reference* to provide the best-performing implementation of atomic operations for the non-atomic object type. All atomic operations performed through an *atomic reference* on a referenced non-atomic object are atomic with respect to any other *atomic reference* that references the same object, as defined by equality of pointers to that object. The intent is for atomic operations to directly update the referenced object. An *atomic reference constructor* may acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. Such *atomic reference* objects are not lock free and not address free. When such a resource is necessary, subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource.

Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with an *atomic* object.

An object may be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively *atomic* would incur an unnecessary performance penalty.

3.2. Atomic Operations on Members of a Very Large Array

High-performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

4. *Reference-ability* Constraints

An object referenced by an *atomic reference* must satisfy possibly architecture-specific constraints. For example, the object might need to be properly aligned in memory or may not be allowed to reside in GPU register memory. We do not enumerate all potential constraints or specify behavior when these constraints are violated. It is a quality-of-implementation issue to generate appropriate information when constraints are violated.

> Note: Whether an implementation of `atomic<T>` is lock free, does not necessarily constrain whether the corresponding implementation of `atomic_ref<T>` is lock free.

5. Concern with `atomic<T>` and padding bits in `T`

A concern has been discussed for `atomic<T>` where `T` is a class type that contains padding bits and how construction and `compare_exchange` operations are effected by the value of those padding bits. We require that the resolution of padding bits follow [P0528r2].

6. Questions and Concerns

1. Should `is_lock_free()` be consistent across objects of the same type?

   There is concern that if this function is not consistent (for both `atomic<T>` and *atomic reference* of T) then there is no reasonable way to choose an algorithm depending on that property.

2. Should the wording of `atomic<T>` be rewritten in terms of an *atomic reference*?

   Result of SG1 poll

SF	F	N	A	SA
4	3	13	0	0

   Rewriting `atomic<T>` would require revisiting the contentious issue of `atomic<T>` having an exposition only member of type `T`. While there is strong support for this rewrite in SG1, we decided to limit the scope of this paper to *atomic reference* to reduce the potential of unnecessary conflict. We will attempt this rewrite in a future paper.

3. Does *atomic reference* of T need C compatibility?

   We decided to leave C compatibility out of this paper, but that it needs to be address in a future paper.

4. Should *atomic reference* of T be named `atomic_ref<T>` or `atomic<T&>`?

   Result of SG1 poll

SF	F	N	A	SA
0	7	11	3	1

   Those against `atomic<T&>` raised the concern that it allows dangerous errors to creep into generic code, which requires users to be aware of this edge case to avoid. Also, after an `atomic<T>` is constructed it does not have data races with other objects, while an *atomic reference* of T does. Furthermore `atomic<T&>` does not have *volatile* member functions. Consequently, `atomic<T&>` is a specialization of `atomic<T>` with weaker guarantees.

   The arguments for `atomic<T&>` is that it is more concise and reduces the vocabulary terms that a user needs to know.

   We decided to keep the name of an *atomic reference* of T as `atomic_ref<T>` for two reasons. First, using the name `atomic_ref<T>` removes any possibility of impacting existing generic code which uses `atomic<T>`. Second, when trying to create wording for `atomic<T&>` the specializations had a distinct `vector<bool>` feel where each specialization needed to walk back from guarantees made by the primary template. In particular, the `atomic<T&>` specializations would be unable to use the phrase *"Descriptions are provide below only for members that differ from the primary template"*.

§ 7. Proposal

The proposed changes are relative to the working draft of the standard as of [N4727].

> Text in blockquotes is not proposed wording

> The ❡ character is used to denote a placeholder section number which the editor shall determine.

> Apply the following changes to 32.2.❡ [atomics.syn]:

```
namespace std {

// 3.□ atomic ref
template<class T> struct atomic_ref;
// 3.□ atomic ref partial specialization for pointers
template<class T> struct atomic_ref<T *>;

}
```

> Add a new section [atomics.ref.generic] after [atomics.types.generic]

```
template<class T> struct atomic_ref {
private:
  T * ptr_; // exposition only
public:
  using value_type = T;
  static constexpr bool is_always_lock_free = implementation-defined;
  static constexpr size_t required_alignment = implementation-defined;

  atomic_ref() = delete;
  atomic_ref& operator=(const atomic_ref&) = delete;

  explicit atomic_ref(T&);
  atomic_ref(const atomic_ref&);

  T operator=(T) const noexcept;
  operator T() const noexcept;

  bool is_lock_free() const noexcept;
  void store(T , memory_order = memory_order_seq_cst) const noexcept;
  T load(memory_order = memory_order_seq_cst) const noexcept;
  T exchange(T , memory_order = memory_order_seq_cst) const noexcept;
  bool compare_exchange_weak(T&, T, memory_order, memory_order) const noexcept;
  bool compare_exchange_strong(T&, T, memory_order, memory_order) const noexcept;
  bool compare_exchange_weak(T&, T, memory_order = memory_order_seq_cst) const noexcept;
  bool compare_exchange_strong(T&, T, memory_order = memory_order_seq_cst) const noexcept;
};
```

The template argument for T shall be trivially copyable [basic.types].

Descriptions are provided below.

> Add a new subsection [atomics.ref.operations] to [atomics.ref.generic]

static constexpr size_t is_always_lockfree;

The static data member `is_always_lock_free` is true if the `atomic_ref` type's operations are always lock-free, and false otherwise.

static constexpr size_t required_alignment;

The required alignment of an object to be referenced by an atomic reference, which is at least `alignof(T)`.

[*Note:* An implementation may require an object to be referenced by an *atomic reference* to have stricter alignment [basic.align] than other objects of type T. Further, whether operations on a *atomic reference* of T are lock-free may depend on the alignment of the referenced object. For example, an implementation may support lock-free operations on `std::complex<double>` only if aligned to `2*alignof(double)`. - *end note*]

atomic_ref(T& obj);

Requires: The referenced non-atomic object has to be aligned to `required_alignment`.

Effects: Constructs an atomic reference that references the non-atomic object.

Throws: Nothing.

Remarks: The lifetime (6.8) of `*this` shall not exceed the lifetime of the referenced non-atomic object. While any `atomic_ref` instance exists that references the object all accesses of that object shall exclusively occur through those `atomic_ref` instances.

If the referenced *object* is of a class or aggregate type, then members of that object shall not be concurrently referenced by an `atomic_ref` object.

Atomic operations applied to object through a referencing atomic reference are atomic with respect to atomic operations applied through any other atomic reference that references that object.

[*Note*: The constructor may acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations applied to the referenced non-atomic object. - *end note*]

atomic_ref(const atomic_ref& ref);

Effects: Construct an atomic reference that references the non-atomic object referenced by the given `atomic_ref`.

T operator=(T desired) const noexcept; *Effects:* Equivalent to store(desired)

Returns: desired

operator T() const noexcept;

Effects: Equivalent to: return load();

bool is_lock_free() const noexcept;

Returns: true if the object's operations are lock-free, false otherwise.

void store(T desired, memory_order order = memory_order_seq_cst) const noexcept; *Requires:* The order argument shall not be memory_order_consume, memory_order_acquire, nor memory_order_acq_rel.

*Effects:*Atomically replaces the value pointed to by `ptr_` with the value of desired. Memory is affected according to the value of order.

void load(memory_order order = memory_order_seq_cst) const noexcept; *Requires:* The order argument shall not be memory_order_release nor memory_order_acq_rel.

Effects: Memory is affected according to the value of order.

Returns: Atomically returns the value pointed to by `ptr_`.

exchange(T desired, memory_order order = memory_order_seq_cst) noexcept;

Effects: Atomically replaces the value pointed to by `ptr_` with desired. Memory is affected according to the value of order. These operations are atomic read-modify-write operations [intro.multithread].

Returns: Atomically returns the value pointed to by `ptr_` immediately before the effects.

bool compare_exchange_weak(T& expected, T desired, memory_order success, memory_order failure) const noexcept;

bool compare_exchange_strong(T& expected, T desired, memory_order success, memory_order failure) const noexcept;

bool compare_exchange_weak(T& expected, T desired, memory_order order = memory_order_seq_cst) const noexcept;

bool compare_exchange_strong(T& expected, T desired, memory_order order = memory_order_seq_cst) const noexcept;

Requires: The failure argument shall not be memory_order_release nor memory_order_acq_rel.

Effects: Retrieves the value in expected. It then atomically compares the contents of the memory pointed to by `ptr_` for equality with that previously retrieved from expected, and if true, replaces the contents of the memory pointed to by `ptr_` with that in desired. If and only if the comparison is true, memory is affected according to the value of success, and if the comparison is false, memory is affected according to the value of failure. When only one memory_order argument is supplied, the value of success is order, and the value of failure is order except that a value of memory_order_acq_- rel shall be replaced by the value memory_order_acquire and a value of memory_order_release shall be replaced by the value memory_order_relaxed. If and only if the comparison is false then, after the atomic operation, the contents of the memory in expected are replaced by the value read from the memory pointed to by `ptr_` during the atomic comparison. If the operation returns true, these operations are atomic read-modify-write operations (6.8.2) on the memory pointed to by `ptr_`. Otherwise, these operations are atomic load operations on that memory.

Returns: The result of the comparison.

Remarks: A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by expected and `ptr_` are equal, it may return false and store back to expected the same memory contents that were originally there. [Note: This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare- and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — end note]

[*Note:* The memcpy and memcmp semantics of the compare-and-exchange operations may result in failed comparisons for values that compare equal with operator== if the underlying type has padding bits, trap bits, or alternate representations of the same value. Notably, on implementations conforming to ISO/IEC/IEEE 60559, floating-point -0.0 and +0.0 will not compare equal with memcmp but will compare equal with operator==, and NaNs with the same payload will compare equal with memcmp but will not compare equal with operator==. — *end note*]

> Add a new subsection [atomics.ref.int] following the [atomics.ref.operations] subsection.

```
template<> struct atomic_ref<integral> {
private:
  integral* ptr_; // exposition only
public:
  using value_type = integral;
  using difference_type = value_type;
  static constexpr bool is_always_lock_free = implementation-defined;
  static constexpr size_t required_alignment = implementation-defined;

  atomic_ref() = delete;
  atomic_ref& operator = (const atomic_ref&) = delete;

  explicit atomic_ref(integral&);
  atomic_ref(const atomic_ref&);

  integral operator=(integral) const noexcept;
  operator integral () const noexcept;

  bool is_lock_free() const noexcept;
  void store(integral , memory_order = memory_order_seq_cst) const noexcept;
  integral load(memory_order = memory_order_seq_cst) const noexcept;
  integral exchange(integral , memory_order = memory_order_seq_cst) const noexcept;
  bool compare_exchange_weak(integral& , integral , memory_order , memory_order) const noex
cept;
  bool compare_exchange_strong(integral& , integral  , memory_order , memory_order) const n
oexcept;
  bool compare_exchange_weak(integral& , integral  , memory_order = memory_order_seq_cst) c
onst noexcept;
  bool compare_exchange_strong(integral&, integral , memory_order = memory_order_seq_cst) c
onst noexcept;

  integral fetch_add(integral , memory_order = memory_order_seq_cst) const noexcept;
  integral fetch_sub(integral , memory_order = memory_order_seq_cst) const noexcept;
  integral fetch_and(integral , memory_order = memory_order_seq_cst) const noexcept;
  integral fetch_or(integral , memory_order = memory_order_seq_cst) const noexcept;
  integral fetch_xor(integral , memory_order = memory_order_seq_cst) const noexcept;

  integral operator++(int) const noexcept;
  integral operator--(int) const noexcept;
  integral operator++() const noexcept;
  integral operator--() const noexcept;
  integral operator+=(integral) const noexcept;
  integral operator-=(integral) const noexcept;
  integral operator&=(integral) const noexcept;
  integral operator|=(integral) const noexcept;
  integral operator^=(integral) const noexcept;
};
```

Descriptions are provide below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 129 [atomics.types.int].

integral fetch_*key*(integral operand, memory_order order = memory_order_seq_cst) const noexcept;

Effects: Atomically replaces the value pointed to by `ptr_` with the result of the computation applied to the value pointed to by `ptr_` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations (6.8.2).

Returns: Atomically, the value pointed to by `ptr_` immediately before the effects.

Remarks: For signed integer types, arithmetic is defined to use two's complement representation. There are no undefined results.

integral operator++() const noexcept;

Effects: Equivalent to: `return fetch_add(1) + 1;`

integral operator--() const noexcept;

Effects: Equivalent to: `return fetch_sub(1) - 1;`

integral operator++(int) const noexcept;

Effects: Equivalent to: `return fetch_add(1);`

integral operator--(int) const noexcept;

Effects: Equivalent to: `return fetch_sub(1);`

integral operator *op*=(integral operand) const noexcept;

Effects: Equivalent to: `return fetch_key(operand) op operand;`

> Add a new subsection [atomics.ref.float] following the [atomics.ref.int] subsection.

```
template<> struct atomic_ref<floating-point> {
private:
  floating-point* ptr_; // exposition only
public:
  using value_type = floating-point;
  using difference_type = value_type;
  static constexpr bool is_always_lock_free = implementation-defined;
  static constexpr size_t required_alignment = implementation-defined;

  atomic_ref() = delete;
  atomic_ref& operator = (const atomic_ref&) = delete;

  explicit atomic_ref(floating-point&) noexcept;
  atomic_ref(const atomic_ref&);

  floating-point operator=(floating-point) noexcept;
  operator floating-point () const noexcept;

  bool is_lock_free() const noexcept;
  void store(floating-point , memory_order = memory_order_seq_cst) const noexcept;
  floating-point load(memory_order = memory_order_seq_cst) const noexcept;
  floating-point exchange(floating-point , memory_order = memory_order_seq_cst) const noexc
ept;
  bool compare_exchange_weak(floating-point& , floating-point , memory_order , memory_order
) const noexcept;
  bool compare_exchange_strong(floating-point& , floating-point  , memory_order , memory_or
der) const noexcept;
  bool compare_exchange_weak(floating-point& , floating-point  , memory_order = memory_orde
r_seq_cst) const noexcept;
  bool compare_exchange_strong(floating-point&, floating-point , memory_order = memory_orde
r_seq_cst) const noexcept;

  floating-point fetch_add(floating-point , memory_order = memory_order_seq_cst) const noex
cept;
  floating-point fetch_sub(floating-point , memory_order = memory_order_seq_cst) const noex
cept;

  floating-point operator+=(floating-point) const noexcept;
  floating-point operator-=(floating-point) const noexcept;
};
```

Descriptions are provide below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 129 [atomics.types.int].

floating-point fetch_*key*(floating-point operand, memory_order order = memory_order_seq_cst) const noexcept;

Effects: Atomically replaces the value pointed to by `ptr_` with the result of the computation applied to the value pointed to by `ptr_` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations (6.8.2).

Returns: Atomically, the value pointed to by `ptr_` immediately before the effects.

Remarks: If the result is not a representable value for its type (8.1) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point should conform to the std::numeric_limits traits associated with the floating- point type (21.3.2). The floating-point environment (29.4) for atomic arithmetic operations on floating-point may be different than the calling thread's floating-point environment.

floating-point operator *op*=(floating-point operand) const noexcept;

Effects: Equivalent to: `return fetch_key(operand) op operand;`

> Add a new subsection [atomics.ref.pointer] following the [atomics.ref.float] subsection.

```
template<class T> struct atomic_ref<T*> {
private:
  T** ptr_; // exposition only
public:
  using value_type = T*;
  using difference_type = ptrdiff_t;
  static constexpr bool is_always_lock_free = implementation-defined;
  static constexpr size_t required_alignment = implementation-defined;

  atomic_ref() = delete;
  atomic_ref& operator = (const atomic_ref&) = delete;

  explicit atomic_ref(T*&);
  atomic_ref(const atomic_ref&);

  T* operator=(T*) const noexcept;
  operator T* () const noexcept;

  bool is_lock_free() const noexcept;
  void store(T* , memory_order = memory_order_seq_cst) const noexcept;
  T* load(memory_order = memory_order_seq_cst) const noexcept;
  T* exchange(T* , memory_order = memory_order_seq_cst) const noexcept;
  bool compare_exchange_weak(T*& , T* , memory_order , memory_order) const noexcept;
  bool compare_exchange_strong(T*& , T*  , memory_order , memory_order) const noexcept;
  bool compare_exchange_weak(T*& , T*  , memory_order = memory_order_seq_cst) const noexcep
t;
  bool compare_exchange_strong(T*&, T* , memory_order = memory_order_seq_cst) const noexcep
t;

  T* fetch_add(difference_type , memory_order = memory_order_seq_cst) const noexcept;
  T* fetch_sub(difference_type , memory_order = memory_order_seq_cst) const noexcept;

  T* operator++(int) const noexcept;
  T* operator--(int) const noexcept;
  T* operator++() const noexcept;
  T* operator--() const noexcept;
  T* operator+=(difference_type) const noexcept;
  T* operator-=(difference_type) const noexcept;
};
```

Descriptions are provide below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 130 [atomics.types.pointer].

T* fetch_*key*(T* operand, memory_order order = memory_order_seq_cst) const noexcept;

Requires: T shall be an object type, otherwise the program is ill-formed

Effects: Atomically replaces the value pointed to by `ptr_` with the result of the computation applied to the value pointed to by `ptr_` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations (6.8.2).

Returns: Atomically, the value pointed to by `ptr_` immediately before the effects.

Remarks: The result may be an undefined address, but the operations otherwise have no undefined behavior.

T* operator++() const noexcept;

Effects: Equivalent to: `return fetch_add(1) + 1;`

T* operator--() const noexcept;

Effects: Equivalent to: `return fetch_sub(1) - 1;`

T* operator++(int) const noexcept;

Effects: Equivalent to: `return fetch_add(1);`

T* operator--(int) const noexcept;

Effects: Equivalent to: `return fetch_sub(1);`

T* operator *op*=(T* operand) const noexcept;

Effects: Equivalent to: `return fetch_key(operand) op operand;`

8. Feature Testing

The `__cpp_lib_atomic_ref` feature test macro should be added.

References

Informative References

[N4727]
Richard Smith. Working Draft, Standard for Programming Language C++. URL: https://wg21.link/n4727

[P0019r3]
H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. 14 October 2016. URL: https://wg21.link/p0019r3

[P0019r4]
H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. 9 November 2016. URL: https://wg21.link/p0019r4

[P0019r5]
H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. URL: https://wg21.link/p0019r5

[P0019r6]
H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. URL: https://wg21.link/p0019r6

[P0020r5]
H. Carter Edwards, Hans Boehm, Olivier Giroux, JF Bastien, James Reus. Floating Point Atomic. URL: https://wg21.link/p0020r5

[P0558r1]
Resolving atomic named base class inconsistencies. URL: https://wg21.link/p0558r1